LSTM Training on AMD Developer Cloud

LSTM Training on AMD Developer Cloud

Developer Instructions — Orbitra Reroute

Project: Orbitra Reroute (AMD Hackathon Act III) **Task:** Train the PyTorch LSTM blockage classifier on an AMD GPU and hand back `models/lstm_blockage.pt`

Overview

You will spin up an AMD Developer Cloud GPU instance, implement and run the LSTM training script, and copy the resulting model artifact back to the repo. The rest of the app already knows how to load and serve predictions from it — no other changes are needed outside the two files listed below.

Files you will modify:

- `scripts/train_model.py` — implement `train_lstm()`
- `app/risk/ml_model.py` — add LSTM loader + inference path

Artifact to deliver: `models/lstm_blockage.pt`

Step 1 — Get AMD Developer Cloud Access

1. Go to **cloud.amd.com** and sign in (or create an account)
2. Request a GPU instance — ask for **MI300X** (preferred) or MI250X
3. Once provisioned, SSH in:

```
ssh -i your-key.pem user@<instance-ip>
```

Step 2 — Verify ROCm and GPU

```
rocm-smi                  # should show AMD GPU(s)
rocminfo | grep gfx       # should show gfx942 (MI300X) or gfx90a (MI250X)
```

Step 3 — Clone the Repo and Set Up Environment

```
git clone <repo-url>
cd "AMD Hackathon"

# Install uv package manager if not present
curl -Lsf https://astral.sh/uv/install.sh | sh

# Create venv and install base dependencies
uv sync

# Install ROCm-compatible PyTorch
# For ROCm 6.1:
uv pip install torch --index-url https://download.pytorch.org/whl/rocm6.1

# For ROCm 6.2 (use whichever matches your instance):
# uv pip install torch --index-url https://download.pytorch.org/whl/rocm6.2
```

Verify PyTorch sees the AMD GPU:

```
python -c "import torch; print(torch.cuda.is_available());
        print(torch.cuda.get_device_name(0))"
# Expected output:
# True
# AMD Instinct MI300X  (or similar)
```

Step 4 — Implement `train_lstm()` in `scripts/train_model.py`

Find the `train_lstm()` function (around line 76). It currently raises `NotImplementedError`. Replace the entire function body with the following:

```
def train_lstm() -> None:
    """PyTorch LSTM on AMD ROCm GPU. Saves to models/lstm_blockage.pt."""
    import csv
    import torch
    import torch.nn as nn
    from torch.utils.data import DataLoader, TensorDataset
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Training on: {device}")
if device.type == "cuda":
    print(f"  GPU: {torch.cuda.get_device_name(0)}")

# Load seed training data (same source as XGBoost path)
path = DATA_DIR / "portwatch_targets.csv"
if not path.exists():
    print(f"ERROR: {path} not found.")
    sys.exit(1)

with open(path, newline="", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))

X, y = [], []
for row in rows:
    vessel_count = float(row.get("vessel_count", 100) or 100)
    goldstein    = 0.0   # placeholder — matches XGBoost training data
    news_risk    = 5.0   # placeholder — matches XGBoost training data
    X.append([vessel_count, goldstein, news_risk])
    y.append(float(row.get("target", 0) or 0))

# Shape: (samples, seq_len=1, features=3)
X_tensor = torch.tensor(X, dtype=torch.float32).unsqueeze(1).to(device)
y_tensor = torch.tensor(y, dtype=torch.float32).unsqueeze(1).to(device)

dataset = TensorDataset(X_tensor, y_tensor)
loader  = DataLoader(dataset, batch_size=8, shuffle=True)

class LSTMClassifier(nn.Module):
    def __init__(self, input_size=3, hidden_size=64, num_layers=1):
        super().__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers,
                            batch_first=True)
        self.head = nn.Linear(hidden_size, 1)

    def forward(self, x):
        _, (h_n, _) = self.lstm(x)
        return self.head(h_n[-1])

model     = LSTMClassifier().to(device)
criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

EPOCHS = 50
for epoch in range(1, EPOCHS + 1):
    model.train()
    epoch_loss = 0.0
    for xb, yb in loader:
        optimizer.zero_grad()
        loss = criterion(model(xb), yb)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()
```

```
        if epoch % 10 == 0:
            avg = epoch_loss / len(loader)
            print(f"  Epoch {epoch}/{EPOCHS}  loss={avg:.4f}")

    MODELS_DIR.mkdir(exist_ok=True)
    out = MODELS_DIR / "lstm_blockage.pt"
    torch.save(model.state_dict(), out)
    print(f"\nSaved LSTM model → {out}")
    if device.type == "cuda":
        print(f"Device used: {torch.cuda.get_device_name(0)}")
```

Step 5 — Update `app/risk/ml_model.py`

5a — Update `_load_model()` to load the LSTM artifact

Find the `_load_model()` function and replace it entirely with:

```
def _load_model() -> Any:
    global _model
    if _model is not None:
        return _model
    if _XGBOOST_PATH.exists():
        import joblib
        _model = joblib.load(_XGBOOST_PATH)
        logger.info("Loaded XGBoost classifier from %s", _XGBOOST_PATH)
        return _model
    if _LSTM_PATH.exists():
        import torch
        import torch.nn as nn

        class LSTMClassifier(nn.Module):
            def __init__(self, input_size=3, hidden_size=64, num_layers=1):
                super().__init__()
                self.lstm = nn.LSTM(input_size, hidden_size, num_layers,
                                    batch_first=True)
                self.head = nn.Linear(hidden_size, 1)
            def forward(self, x):
                _, (h_n, _) = self.lstm(x)
                return self.head(h_n[-1])

        m = LSTMClassifier()
        m.load_state_dict(torch.load(_LSTM_PATH, map_location="cpu"))
        m.eval()
        _model = ("lstm", m)
        logger.info("Loaded LSTM classifier from %s", _LSTM_PATH)
        return _model
    logger.warning(
        "No trained model found at %s. Run `make train`. Using fallback rules.",
```

```
        _XGBOOST_PATH,
    )
    return None
```

5b — Update `predict_blockage()` to handle LSTM

Replace the entire `predict_blockage()` function with:

```
async def predict_blockage(features: ChokePointFeatures) -> BlockagePrediction:
    """Run the ML classifier. Falls back to rules if model not loaded."""
    import torch

    loaded = _load_model()

    vessel_count = features.vessel_count    if features.vessel_count    is not None else 100
    goldstein    = features.goldstein_score if features.goldstein_score is not None else 0.0
    news_risk    = features.news_risk_score if features.news_risk_score is not None else 5.0

    if loaded is None:
        prob = 0.0
        if news_risk  >= 7:  prob += 0.3
        if goldstein  <= -3: prob += 0.25
        if features.vessel_count is not None and features.vessel_count < 80:
            prob += 0.2
        prob = min(prob, 1.0)
        model_used = "fallback_rules"

    elif isinstance(loaded, tuple) and loaded[0] == "lstm":
        _, lstm_model = loaded
        x = torch.tensor(
            [[[vessel_count, goldstein, news_risk]]], dtype=torch.float32
        )
        with torch.no_grad():
            logit = lstm_model(x).item()
        prob = float(torch.sigmoid(torch.tensor(logit)).item())
        model_used = "lstm"

    else:
        # XGBoost path
        prob = float(loaded.predict_proba(
            [[vessel_count, goldstein, news_risk]]
        )[0][1])
        model_used = "xgboost"

    classification = (
        BlockageClassification.BLOCKED
        if prob >= 0.5 else BlockageClassification.NOT_BLOCKED
    )
    confidence = min(abs(prob - 0.5) * 2, 1.0)

    logger.info(
        "predict_blockage: chokepoint=%s prob=%.3f model=%s",
```

```
        features.chokepoint, prob, model_used,
    )

    return BlockagePrediction(
        chokepoint=features.chokepoint,
        probability=prob,
        classification=classification,
        model_used=model_used,
        confidence=confidence,
        features_used=features,
    )
```

Step 6 — Run Training

```
uv run python scripts/train_model.py --model lstm
```

Expected output:

```
Training lstm blockage classifier...
Training on: cuda
  GPU: AMD Instinct MI300X
  Epoch 10/50  loss=0.6821
  Epoch 20/50  loss=0.6134
  Epoch 30/50  loss=0.5502
  Epoch 40/50  loss=0.4891
  Epoch 50/50  loss=0.4203

Saved LSTM model → /path/to/models/lstm_blockage.pt
Device used: AMD Instinct MI300X
```

Step 7 — Copy Artifact Back to the Repo

From your **local machine** (not the cloud instance):

```
scp -i your-key.pem \
  user@<instance-ip>:/path/to/AMD\ Hackathon/models/lstm_blockage.pt \
  "./models/lstm_blockage.pt"
```

Then commit:

```
git add models/lstm_blockage.pt
git commit -m "Add LSTM blockage classifier trained on AMD MI300X"
git push
```

Step 8 — Verify Locally (No GPU Needed)

The artifact loads on CPU — no ROCm required to run inference:

```
python -c "
from app.risk.ml_model import _load_model
m = _load_model()
print(m)
# Expected: ('lstm', LSTMClassifier())
"
```

Then run the project smoke tests:

```
make smoke
# All suites should still PASS
```

Notes

- **Model priority:** If both `blockage_classifier.pkl` (XGBoost) and `lstm_blockage.pt` (LSTM) exist, XGBoost takes priority. To force the LSTM path, rename or delete `models/blockage_classifier.pkl`.
- **Inference is CPU-portable:** The `.pt` artifact was saved with `map_location="cpu"` so it runs on any machine without ROCm.
- **Stop the cloud instance** as soon as training completes to avoid unnecessary charges.
- **Feature placeholders:** `goldstein_score` and `news_risk_score` use seed values (0.0 and 5.0) in training, consistent with the XGBoost model. Both models will be retrained with real GDELT features in a future iteration.